ACADEMIC EXERCISE REPORT

Exercise 2: Doctors Database System

SQLite3 Low-Level CRUD Operations & Flask Web Application

Subject: Python Application & Web Development

Date: October 2026

GitHub Repository: quang-tn-beyond/Exercise_2_Doctors_Database

1. Goals and Purpose of the Exercise

The objective of Exercise 2 is to design, implement, and integrate an end-to-end database-backed software system for medical personnel management. The assignment comprises two tightly linked components:

- **Exercise 2-1 (Core Database Engine & CRUD Operations):** Focuses on using Python's built-in `sqlite3` module to create a local relational database for storing doctor records. The goal is to author modular, reusable functions executing core CRUD operations (**C**reate, **R**ead, **U**pdate, **D**elete) with strict transaction management and SQL safety principles (preventing SQL injection via parameterized statements).
- **Exercise 2-2 (Web Application Interface):** Focuses on developing a modern, responsive web application using the **Flask** micro-framework. The application connects directly to the SQLite database established in Exercise 2-1, providing an intuitive Graphical User Interface (GUI) where hospital administrators can submit doctor records through validated HTML forms and inspect structured tabular views across dedicated web pages.

> **Project Architecture Strategy:** To adhere to software engineering best practices (Separation of Concerns), the codebase is organized as two distinct sub-projects under a unified repository: `Exercise_2_1/` contains zero-dependency database scripts, while `Exercise_2_2/` encapsulates the Flask web server, helper modules, and Jinja2 templates.

2. Detailed Description of the Algorithm and Approach

2.1. Database Schema Specification

Both exercises interact with the `doctors.db` SQLite database. A table named `doctors` was designed with the following relational schema:

Column Name	Data Type	Constraints	Description
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique auto-incrementing doctor identifier
`first_name`	TEXT	NOT NULL	Doctor's given name
`last_name`	TEXT	NOT NULL	Doctor's family name
`joining_date`	TEXT	NOT NULL	ISO 8601 formatted date string (`YYYY-MM-DD`)
`salary`	REAL	NOT NULL	Doctor's monthly salary (floating point numeric)
`address`	TEXT	NULLABLE	Clinic or residential address

2.2. Exercise 2-1: Algorithmic Approach & CRUD Design

In `Exercise_2_1/doctors_db.py`, database interactions are structured using Python's context manager (`with sqlite3.connect(...) as conn`). This guarantees that connections are closed deterministically and transactions are committed automatically upon successful block execution, or rolled back if an exception occurs.

1. **Table Creation (`create_table`):** Executes DDL query `CREATE TABLE IF NOT EXISTS doctors (...)` ensuring idempotency without errors on subsequent runs.
2. **Record Insertion (`insert_doctor`):** Executes a parameterized `INSERT INTO doctors VALUES (?, ?, ?, ?, ?)` statement using tuple binding to prevent SQL injection vulnerabilities. Returns `cursor.lastrowid`.
3. **Data Querying (`get_all_doctors` & `get_doctor_by_id`):** Executes `SELECT * FROM doctors`, returning a list of rows or a single record via `fetchone()`.
4. **Record Modification (`update_doctor`):** Executes parameterized `UPDATE doctors SET ... WHERE id = ?`, returning `cursor.rowcount` to verify modification.
5. **Record Removal (`delete_doctor`):** Executes parameterized `DELETE FROM doctors WHERE id = ?`.
6. **Formatted Console Output (`display_doctors`):** Formats doctor records into aligned columns with currency notation (`{:,.2f}`).

2.3. Exercise 2-2: Flask Web Architecture (MVC Pattern)

The web application in `Exercise_2_2` is structured following the Model-View-Controller (MVC) architectural pattern:

- **Model (`db_helper.py`):** Encapsulates all data access logic. Configures `conn.row_factory = sqlite3.Row`, enabling column access by attribute name (e.g., `doctor['first_name']`) within Jinja2 templates.
- **Controller (`app.py`):** Routes client HTTP requests, validates incoming form payloads, coordinates with `db_helper`, and directs responses using the *Post/Redirect/Get (PRG)* pattern to prevent accidental duplicate form submissions upon page reload.
- **View (`templates/`):** Rendered with Bootstrap 5 for clean aesthetics and responsive mobile-first grid layouts.
  - `base.html` : Shared layout containing top navbar, alert container, and footer.
  - `doctor_list.html` : Tabular view listing all doctors with formatted salaries, action buttons (Edit, Delete), and a "+ Add New Doctor" CTA.
  - `add_doctor.html` : Dual-purpose form handling both *New Creation* and *Existing Record Updates*.

3. Program Execution Results & Verification

3.1. Exercise 2-1: Execution Output Verification

The standalone script was executed directly from the terminal via `python doctors_db.py`. The program executed all 6 operational steps without runtime errors:

```
================================================================
  STEP 1: CREATE DATABASE & TABLE
================================================================
[OK] Table 'doctors' is ready.

================================================================
  STEP 2: INSERT SAMPLE DOCTORS
================================================================
  [OK] Inserted: John Smith  (ID: 1)
  [OK] Inserted: Sarah Johnson  (ID: 2)
  [OK] Inserted: Michael Williams  (ID: 3)
  [OK] Inserted: Emily Brown  (ID: 4)
  [OK] Inserted: David Jones  (ID: 5)

================================================================
  STEP 3: SELECT ALL DOCTORS
================================================================
ID    First Name      Last Name       Joining Date            Salary Address
------------------------------------------------------------------------------
1     John            Smith           2020-03-15           75,000.00 123 Main St, New York
2     Sarah           Johnson         2019-07-22           82,000.00 456 Oak Ave, Chicago
3     Michael         Williams        2021-01-10           69,000.00 789 Pine Rd, Houston
4     Emily           Brown           2022-06-05           91,000.00 321 Elm St, San Francisco
5     David           Jones           2023-11-18           65,000.00 654 Maple Dr, Boston

================================================================
  STEP 4: UPDATE DOCTOR (ID = 2)
================================================================
  [OK] Doctor ID 2 updated successfully.
  Updated record:
ID    First Name      Last Name       Joining Date            Salary Address
------------------------------------------------------------------------------
2     Sarah           Johnson-Lee     2019-07-22           95,000.00 100 Lake Shore, Chicago

================================================================
  STEP 5: DELETE DOCTOR (ID = 5)
================================================================
  [OK] Doctor ID 5 deleted successfully.

================================================================
  STEP 6: FINAL STATE OF DATABASE
================================================================
ID    First Name      Last Name       Joining Date            Salary Address
------------------------------------------------------------------------------
1     John            Smith           2020-03-15           75,000.00 123 Main St, New York
2     Sarah           Johnson-Lee     2019-07-22           95,000.00 100 Lake Shore, Chicago
3     Michael         Williams        2021-01-10           69,000.00 789 Pine Rd, Houston
4     Emily           Brown           2022-06-05           91,000.00 321 Elm St, San Francisco

Done! Database file saved at: doctors.db
```

3.2. Exercise 2-2: Web Application Interface Verification

The Flask server was launched at `http://127.0.0.1:5000`. The views below illustrate the functional web pages and form submission flow:

Doctor List Webpage — http://127.0.0.1:5000/ View Page

Doctor List + Add New Doctor

ID	First Name	Last Name	Joining Date	Salary	Address	Actions
1	John	Smith	2020-03-15	$75,000.00	123 Main St, New York	Edit Delete
2	Sarah	Johnson-Lee	2019-07-22	$95,000.00	100 Lake Shore, Chicago	Edit Delete
3	Michael	Williams	2021-01-10	$69,000.00	789 Pine Rd, Houston	Edit Delete
4	Emily	Brown	2022-06-05	$91,000.00	321 Elm St, San Francisco	Edit Delete
6	**Gregory**	**House**	**2024-01-01**	**$120,000.00**	**Princeton-Plainsboro Hospital**	Edit Delete

Add Doctor Form Webpage — http://127.0.0.1:5000/add Form Page

Add New Doctor

First Name * Gregory

Last Name * House

Joining Date * 2024-01-01

Salary ($) * 120000.00

Address Princeton-Plainsboro Hospital

Cancel **Add Doctor**

3.3. Requirement Fulfillment Matrix

Exercise Specification	Implementation Location	Verification Status
Create database for doctors (id, first name, last name, joining date, salary, address)	`Exercise_2_1/doctors_db.py:create_table()`	✓ **Verified** (Table initialized with required columns)
CRUD operations: Select, Insert, Update, Delete functions	`Exercise_2_1/doctors_db.py`	✓ **Verified** (All 5 functions tested in `main()`)
Flask app collecting user input of doctor info via form	`Exercise_2_2/templates/add_doctor.html`	✓ **Verified** (Validated HTML form posting to `/add`)
Display doctor information on additional web page using SQLite	`Exercise_2_2/templates/doctor_list.html`	✓ **Verified** (Dynamic Jinja2 rendering at `/`)

> **Conclusion:** All technical requirements for both Exercise 2-1 and Exercise 2-2 have been thoroughly implemented, tested, and verified. The software codebase adheres to clean architectural principles, safe SQL parametrization, and modern web usability standards.